

including store login, order flow, rider coordination, scheduled orders, wallet, earnings, profile controls, and the local setup points that matter for a successful deployment.

This document reflects the currently implemented mobile application flow in the local project repository as of June 19, 2026.

CLIENT GUIDE

Store App Operational Flow Guide

Application Scope

Store-side workspace for merchants to manage delivery and pickup order operations.
Includes rider coordination, wallet tracking, earnings visibility, and profile controls.

Document Details

Application: Enatega Deliveries Store App
Current build branding: Cylia Merchant
Prepared: June 19, 2026
Purpose: Module Flow and Local Guide

1. Executive Summary

The Store App is the merchant control surface for the delivery ecosystem. A store logs in with its store account, receives new orders in real time, accepts or rejects them, manages preparation timing, marks orders as ready, confirms rider pickup where needed, and tracks completion through live status updates.

The app is built around five operational pillars: authentication and session control, order management, rider communication, financial tracking, and store profile settings. These pillars work together so that day-to-day store operations happen in one place.

2. Primary User Flow

- ? Login and session start: the merchant logs in using store credentials, the app stores the access token securely, and the user is moved from the login flow to the main workspace.
- ? Main workspace opens: the store lands in a four-tab interface covering Orders, Wallet, Earnings, and Profile.
- ? Orders arrive in real time: a live socket connection listens for new orders and order status changes, so new items appear without manual refresh.
- ? Store processes the order: the merchant can accept or reject orders, set preparation timing, move orders to ready, and continue through pickup and completion flow.
- ? Rider coordination continues fulfillment: delivery orders can surface rider assignment, rider progress, rider chat, and call actions when rider details exist.
- ? Financial tracking and settings remain available: the store can review wallet data, earnings, availability, instant delivery settings, schedule, language, and bank details at any time.

3. Module-by-Module Working Flow

- ? Authentication: Controls secure login and session startup through the store email login endpoint, secure token storage, session restoration on app restart, and automatic logout on session expiry.
- ? Orders: Central workspace for new, active, ready, pickup, completed, and scheduled orders. Status-based tabs keep order work organized.
- ? Real-Time Sync: A socket connection opens after login and listens for new orders, order status changes, rider status changes, and chat messages.
- ? Rider Coordination: Supports delivery-side collaboration through rider assignment, rider progress updates, rider chat, and direct phone call launch.
- ? Wallet: Shows current balance, available withdrawal amount, pending withdrawal requests, and recent transaction history.
- ? Earnings: Shows chart summaries, daily earnings activity, and order-based earning history for selected dates.
- ? Profile and Operations: Lets the store manage availability, instant delivery, work schedule, language, bank information, and logout.
- ? Localization: Currently includes English and French language support.

4. Order Lifecycle in Detail

New Order Stage

Orders first appear in the New Orders tab. The store reviews line items, order type, notes, address details, and total amount, then accepts or rejects the order. Delivery and pickup filters are available.

In Progress Stage

Once accepted, the order moves into the preparation state. The store manages preparation timing here and countdown-related data can remain visible.

Ready Stage

When preparation is complete, the order can move into the Ready state to show that the store has finished preparing it for pickup or dispatch.

Pickup Stage

Pickup-related actions are handled in a dedicated tab. This is where rider arrival, rider status, and pickup confirmation become important, especially for non-instant pickup flow.

Completed Stage

Delivered, cancelled, rejected, and failed outcomes are presented in the completed area so the store has a historical view of finalized activity.

Scheduled Orders

Scheduled orders are intentionally separated from immediate order tabs so upcoming workload can be reviewed without mixing it into live active operations.

5. Real-Time and Delivery Coordination Flow

After login, the mobile app opens a socket connection using the authenticated store session. This lets the store receive event-based updates rather than relying only on manual or timed refreshes.

- ? New order created: the order is inserted into matching New Order lists and the alert loop can be triggered.
- ? Order status updated: the app refreshes the relevant order tabs so the order appears in the correct stage.
- ? Rider status updated: the app refreshes active delivery-related tabs so the store sees the rider's latest state.
- ? Chat message received: the message appears inside the rider chat thread and is merged with previously loaded messages.

Important client note: delivery-side coordination depends on the backend socket service, rider assignment logic, and chat services being available and correctly mapped to the store account and order records.

6. Wallet and Earnings Flow

- ? Wallet shows current balance, available withdrawal amount, pending withdrawal requests, and recent transactions.
- ? The store can submit a withdrawal request directly from the app.
- ? Earnings shows chart data, daily earnings activity, summary drilldowns, and order-based earning history by selected date.

- ? Financial values are formatted using the central currency configuration returned by the backend.

7. Profile and Store Controls

- ? Availability toggle marks the store as available or unavailable.
- ? Instant delivery toggle enables immediate delivery or a day-based delivery time setting.
- ? Work schedule defines active days and time slots.
- ? Bank management stores payout-related account information.
- ? Language selection lets the user switch between supported languages.
- ? Scheduled orders are accessible separately from the profile area.
- ? Logout clears the session and returns the app to the login flow.

8. Local Setup and Deployment Notes

- ? EXPO_PUBLIC_API_BASE_URL is required. The app will not boot without this backend API base URL.
- ? EXPO_PUBLIC_SOCKET_URL is optional. If not provided, the socket origin falls back to the API origin.
- ? EXPO_PUBLIC_SOCKET_PATH is optional and defaults to /socket.io. It must match the backend socket setup.
- ? EXPO_PUBLIC_EAS_PROJECT_ID is used as a fallback for push token registration if not already available in Expo config.
- ? google-services.json is required for the Android Firebase and notification configuration already referenced by the app.
- ? Push token registration is intended for a physical device. Simulator-only testing is limited.

9. Important Client Considerations

- ? The app depends on backend auth, order, wallet, earnings, profile, chat, and socket services.
- ? Real-time order and rider coordination work only when the socket server is available and authenticated correctly.
- ? Push notification registration is built into login preparation, but device permissions must be granted by the user.
- ? Store users must have a valid store-linked profile in the auth response for store-specific real-time filtering to work correctly.
- ? The app currently includes English and French localization support.

10. Recommended Handover Checklist

- ? Confirm production API base URL and socket URL/path.
- ? Confirm store login credentials and store-profile mapping.
- ? Validate real-time new order arrival and rider status events.
- ? Validate order action permissions including accept, reject, ready, and confirm pickup.
- ? Validate wallet balance, transaction history, and withdrawal flow.
- ? Validate earnings chart and date-based detail flow.

- ? Validate bank management, work schedule, language, and availability controls.
- ? Validate push-notification permissions on a physical Android or iOS device.

11. Closing Summary

In practical terms, this Store App is designed to be the merchant's live operations console. It combines order intake, preparation flow, rider-side coordination, finance visibility, and store configuration into one mobile experience.

For the client, the most important success factors are a stable backend connection, correct store account mapping, and fully working real-time services. With those dependencies in place, the app provides a complete store-side workflow for both delivery and pickup operations.